

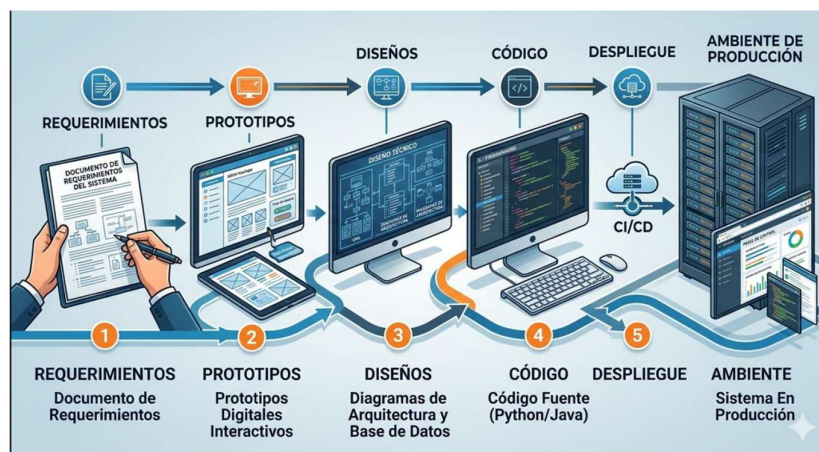
Resumen ISW Parcial 1

Unidad 1 – Ingeniería de Software en contexto

Introducción a la ingeniería de software: ¿Qué es?

Definición de Software

- El software es un conjunto de programas en conjunto con toda la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente (archivos de configuración utilizados para la ejecución, documentos para el usuario, documentos que describen la estructura del sistema), como así también las herramientas utilizadas para la construcción de este.
- Definición integradora de software → información o conocimiento que se presenta en distintos niveles de abstracción, desde los requerimientos del sistema (abstracto), hasta el código que ejecuta el mismo (detallado).
 - o Parte de una idea general (requerimientos) y se va refinando, se van generando artefactos (con distintos niveles de detalle/abstracción) y al final del proceso se obtiene el nivel de detalle que entiende la computadora para ejecutarlo (producto final).
 - o Ejemplo de “artefactos”:
 - Productos de trabajo (que son software)
 - Prototipo de SW
 - CU de software
 - Diseño de interfaz/BD de software
 - Diseño arquitectónico de SW
- Software, en general, es un set de programas y la documentación que acompaña.
- Existen 3 tipos básicos de software:
 - o System Software
 - o Utilitarios
 - o Software de aplicación



5 razones para no comparar software y manufactura:

- ✓ El software es menos predecible
- ✓ No hay producción en masa, casi ningún producto de software es igual a otro
- ✓ No todas las fallas son errores
- ✓ El software no se gasta
- ✓ El software no está gobernado por las leyes de la física

Ingeniería de Software

Definición según autores:

- Parmas → Construcción de software de múltiples versiones por parte de varias personas
- IEEE → es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software
- Sommerville → es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento

Esta ingeniería es importante ya que:

- Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables.
- Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software para los sistemas de software. La mayoría de los costos consisten en cambiar el software después de que se pone en operación.

Además, la ingeniería de software se apoya en varias realidades, entre ellas:

- En primer lugar, se debe hacer un esfuerzo para entender el problema antes de desarrollar una aplicación de software. Es decir, se debe entender el problema y las necesidades del cliente antes de proponer una solución.
- El diseño es una actividad crucial en el desarrollo de un software.
- Realizar un correcto diseño del software, trae consigo dos claros beneficios, calidad de software y facilidad de mantenimiento.

La ingeniería de software nace tras las crisis del software, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollarlo.

Disciplinas que conforman la ingeniería de software

Disciplinas técnicas

- Nos referimos a actividades que aportan al desarrollo de software como **producto**:
 - o Toma de requerimientos
 - o Análisis de requerimiento
 - o Diseño de software
 - o Implementación de software
 - o Prueba de software
 - o Despliegue de un software
 - o Documentación de software
 - o Capacitaciones a usuarios
- Roles de Prueba/Despliegue → ayudan a construir la solución
 - o Devops, analista funcional, analista de requerimientos, programadores, diseñador de BD, diseñador de UX/UI
- Buscan la automatización de la prueba y el despliegue. Se relacionan con las “prácticas continuas” (intento de la ISW de tener más eficiencia y automatización).

Disciplinas de gestión:

- Hacen referencia a actividades de planificación, monitoreo y control del **proyecto** de desarrollo de software:
 - o Planificación de proyecto
 - o Monitoreo y control de proyectos
 - o Proyecto → unidad de gestión de los recursos y las personas para obtener el resultado que esperamos
 - o Posee fecha de inicio y fin, tareas interrelacionadas, evolución gradual
 - o Cada proyecto que se ejecute va a tener una versión diferente de producto
- Se utilizan frameworks ágiles como scrum (te da pautas de cómo vas a supervisar tu proyecto)

Disciplinas de soporte:

- Son disciplinas transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software. Entre ellas se incluyen:
 - o Gestión de Configuración del Software (SCM) → mantener la integridad del producto de SW mientras exista
 - o Toma de métricas → un número, una valoración objetiva (utilizando números)
 - Se mide en “Proceso”, “Proyecto” y “Producto”
 - o Aseguramiento de calidad (QA - Quality Assurance) → realizar una serie de actividades que acompañan la construcción del producto para que el producto satisfaga las necesidades y expectativas por las que fue creado (NO CONFUNDIR CON TESTING)
 - Calidad → nivel de satisfacción, expectativas y necesidades
 - Necesidad → lo que abiertamente manifestamos
 - Expectativa → lo que no manifestamos, pero “esperamos”

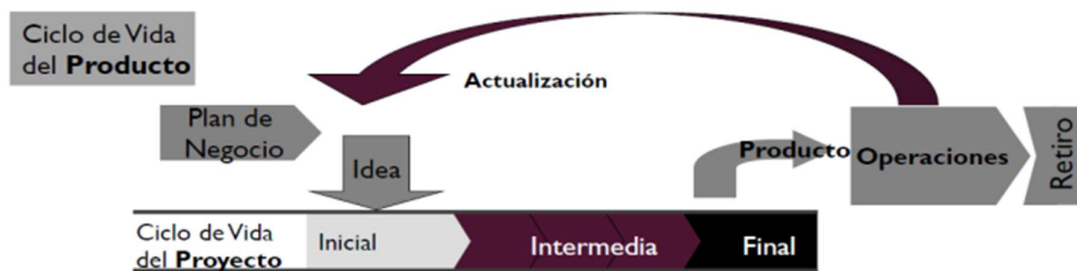
Conceptos – Agilismo, Metodología y Framework

- Agilismo → es un pensamiento o filosofía, NO ES UNA METODOLOGÍA
 - o Conjunto de principios y valores que determinan la manera en que se espera que vos hagas las cosas
 - o Se espera que las personas que están involucradas en el desarrollo de SW sean autogestionadas, independientes, capacitados, motivados
- Metodología → ciencia que estudia la forma de hacer las cosas
- Framework → un marco de trabajo, conjunto de buenas prácticas

Ciclos de vida (Modelos de proceso) y su influencia en la Administración de proyectos de SW

- Se denomina ciclo de vida a una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo. Es una representación simplificada de un proceso, el cuál define elementos del proceso (actividades estructurales, productos del trabajo, tareas, etc.) y el flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos. Son modelos genéricos de los procesos de software; y establecen los criterios que utiliza para determinar si se debe pasar de una tarea a la siguiente. Son independientes de los procedimientos de cada actividad del ciclo de vida.
- Los modelos especifican:
 - o Las fases del proceso → requerimientos, especificación, diseño, etc
 - o El orden en el cual se llevan a cabo

Relación entre el ciclo de vida del proyecto y del producto



- No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar, dependiendo esto de la madurez que tenga el producto en base a las necesidades del mercado.
- Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando. Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos

Tipos de ciclos de vida

- Secuencial → dispone de las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada. Generalmente, el trabajo principal del producto es la documentación del software entre las distintas fases.
- Iterativo/Incremental → aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades. Cada iteración produce un incremento de software funcional potencialmente entregable. El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior.
- Recurso → utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente. Ejemplo: "modelo en espiral" (objetivos, riesgos, desarrollo y prueba, planificar la próxima iteración)

Procesos de Desarrollo Empíricos vs Definidos

Proceso de Desarrollo de Software



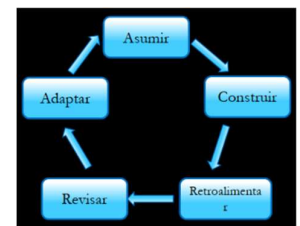
- Un proceso es una secuencia de pasos ejecutados para un propósito dado (IEEE), mientras que un proceso de software se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.
- Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba. El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.

Se tienen 3 factores determinantes del proceso:

- Procedimientos y métodos
- Personas motivadas, capacitadas y con habilidades
- Herramientas y equipos

Procesos definidos vs Procesos empíricos

- Procesos definidos
 - o Asume que podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados.
 - o La administración y control provienen de la predictibilidad del proceso definido
- Procesos empíricos
 - o Asume procesos complicados con variables cambiantes. Cuando se repite el proceso, se pueden llegar a obtener resultados diferentes.
 - o La administración y control es a través de inspecciones frecuentes y adaptaciones
 - o Son procesos que trabajan bien con procesos creativos y complejos.



Criterios para la selección de un modelo de proceso:

1. Si los requerimientos son claros y exhaustivos, en un contexto de baja incertidumbre en el que el cliente no requiere obtener el producto rápidamente, es posible aplicar el modelo secuencial.
2. Si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema, entonces sería conveniente la elección del modelo iterativo- incremental.
3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el modelo recursivo puede ser una buena opción.

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	● Alta certeza (baja incertidumbre). ● La organización tiene una forma tradicional de trabajar. ● No se pueden realizar entregas parciales del software (se debe entregar todo junto). ● Eliminar riesgos de planificación.
Iterativo/incremental	Definidos o Empíricos	● Existe incertidumbre. ● Volatilidad de requerimientos. ● Posibilidad de entregas parciales. ● Uso en etapas tempranas del producto.
Recurso	Definidos	● Mucho control de riesgos en cada iteración. ● No se pueden realizar entregas parciales.

Componentes de un Proyecto de Sistemas de Información

Es un esfuerzo temporal que reúne personas, especialidades y recursos para crear un producto, servicio o resultado único.

Permite adaptar el proceso de desarrollo al contexto y recursos disponibles, definiendo:

- Qué hacer: procesos y tareas.
- Quién lo hace: personal y responsabilidades.
- Cómo controlar: mecanismos para gestionar riesgos y cambios, y evaluar la calidad.

Características

- **Únicos:** El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes.
- **Orientado a objetivos**
- **No ambiguos:** deben ser lo suficientemente claros para guiar el desarrollo del proyecto.
- **Medibles:** es necesario medir el objetivo para determinar el avance sobre el proyecto.
- **Alcanzable:** deben ser factibles de realizarse por el equipo (que el equipo pueda hacerlos).
- **Duración limitada de tiempo:** Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas. Cuando se alcanzan los objetivos del proyecto, este llega a su fin.
- **Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos:** Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Triple restricción – Enfoque tradicional

Todo proyecto está condicionado por tres variables: alcance, tiempo y costo. Su equilibrio afecta directamente la calidad del producto y es responsabilidad del líder del proyecto.

Principio: no es posible fijar arbitrariamente las tres variables; se pueden fijar dos y la tercera debe ajustarse. La calidad no debe sacrificarse para cumplir las restricciones.

Variable	Definición
Alcance	Requerimientos y funcionalidades que tendrá el producto.
Tiempo	Plazo establecido para finalizar el proyecto y entregar el producto.
Costo	Recursos económicos necesarios: personal, equipamiento, infraestructura, etc.

Relación entre variables: cuanto mayor sea el alcance, generalmente mayores serán el tiempo y el costo necesarios.

Problema del enfoque tradicional: el alcance se define al inicio del proyecto y se mantiene relativamente fijo. Si aparecen cambios, pueden aumentar el tiempo y el costo o comprometerse la calidad al reducir actividades como las pruebas.

Diferencia con el enfoque ágil: se mantiene fijo el tiempo y se adapta o negocia el alcance según las necesidades y el feedback del cliente, sin sacrificar la calidad.

Plan de proyecto

El Plan de Proyecto (o Plan de Desarrollo de Software) es un artefacto de gestión que funciona como la hoja de ruta del proyecto. Documenta las decisiones sobre qué se hará, cómo, cuándo y quién será responsable, incluyendo mecanismos para gestionar riesgos, cambios y calidad.

Actividades:

1. Definición del alcance del proyecto.

Concepto	Definición	Se mide contra
Alcance del producto	Funcionalidades y características que debe tener el software para satisfacer los requerimientos del cliente.	ERS (Especificación de Requerimientos).
Alcance del proyecto	Todo el trabajo, y solo el trabajo necesario, para desarrollar y entregar el producto especificado.	Plan de Proyecto.

Idea clave: el alcance del producto define qué debe hacer el software; el alcance del proyecto define qué trabajo debe realizarse para construirlo.

2. Definición de proceso y ciclo de vida.
3. Estimación

4. Gestión de riesgos.

Un riesgo es una amenaza potencial que, si ocurre, puede generar pérdidas o daños y comprometer los objetivos del proyecto.

Tipos de riesgos:

- **Del proyecto:** afectan el calendario o los recursos. Ej.: renuncia de un integrante.
- **Del producto:** afectan la calidad o el rendimiento del software. Ej.: falla de un componente.
- **Organizacionales o de negocio:** afectan a la organización. Ej.: aparición de un producto competidor.

Exposición al riesgo = Probabilidad de ocurrencia × Impacto. Permite valorar y priorizar los riesgos sobre los que se debe actuar.

Actitudes frente al riesgo:

- **Negación:** ignorar su existencia.
- **Reactiva:** actuar recién cuando ocurre.
- **Proactiva:** anticiparse mediante planes de mitigación y contingencia.

Etapas de la gestión de riesgos:

1. **Identificación:** reconocer posibles riesgos.
2. **Análisis:** evaluar su probabilidad e impacto.
3. **Planificación:** definir estrategias para gestionarlos.
4. **Monitorización y control:** hacer seguimiento y reevaluarlos durante todo el proyecto.

Diferencia importante: un plan de **mitigación** busca reducir la exposición al riesgo; un plan de **contingencia** establece cómo actuar si el riesgo ocurre. </>

5. Asignación de recursos.
6. Programación de proyectos.
7. Definición de métricas.

Una **métrica** es una medida numérica que proporciona visibilidad sobre el estado o avance del proyecto, proceso o producto.

- Deben definirse claramente, incluyendo cómo se calculan.
- Deben aportar información útil para el seguimiento y la toma de decisiones.
- En el enfoque tradicional se utilizan para medir y controlar el avance; en ágil, un indicador fundamental del progreso son los incrementos entregados.

Métricas básicas de un proyecto: tamaño del producto, esfuerzo, calendario (tiempo), defectos y costos. </>

Tipo	¿Qué miden?	Objetivo	Ejemplos
Métricas de proyecto	Recursos y avance de un proyecto específico.	Controlar la planificación, los costos y los plazos, detectando desvíos.	Costo estimado vs. real, esfuerzo utilizado, fechas de entrega previstas vs. reales.
Métricas de proceso	El funcionamiento de los procesos de desarrollo de la organización.	Mejorar los procesos a largo plazo.	Desviación promedio de estimaciones, defectos detectados por usuarios, tiempo promedio de planificación.
Métricas de producto	Características técnicas y calidad del software construido.	Evaluar el producto y verificar que cumpla los requisitos de calidad.	Líneas de código, defectos por severidad, cantidad de métodos por clase.

Diferencias importantes para el parcial

- **Proyecto:** son particulares de cada proyecto, generalmente privadas y responsabilidad del Líder de Proyecto.
- **Proceso:** son estratégicas, se obtienen consolidando datos de distintos proyectos y se utilizan a nivel organizacional.
- **Producto:** son particulares del software construido, tienen un enfoque técnico y son responsabilidad de Desarrollo y Testing.

Ejemplo para distinguirlas: si medís los defectos de una aplicación específica, es una métrica de **producto**. Si consolidás los defectos de todas las aplicaciones para evaluar cómo funciona el proceso de desarrollo de la organización, es una métrica de **proceso**.

Para recordar: proyecto = *cómo avanza el trabajo*; proceso = *cómo trabaja la organización*; producto = *cómo es el software construido*.

• Desarrollador

1. Esfuerzo
2. Esfuerzo y duración estimada y actual de una tarea.
3. % de cobertura por el unit test
4. Numero y tipo de defectos encontrados en el unit test.
5. Numero y tipo de defectos encontrados en revisión por pares.

• Organización

1. Tiempo Calendario
2. Performance actual y planificada de esfuerzo.
3. Performance actual y planificada de presupuesto
4. Precisión de estimaciones en Schedule y esfuerzo
5. Defectos en Release

• Equipo de Desarrollo

1. Tamaño del producto
2. Duración estimada y actual entre los hitos más importantes.
3. Niveles de staffing actuales y estimados.
4. Nro. de tareas planificadas y completadas.
5. Distribución del esfuerzo
6. Status de requerimientos.
7. Volatilidad de requerimientos.
8. Nro. de defectos encontrados en la integración y prueba de sistemas.
9. Nro. de defectos encontrados en peer reviews.
10. Status de distribución de defectos.
11. % de test ejecutados

8. Definición de controles.

No silver Bullet (No hay balas de plata)

Fred Brooks plantea que no existe una única tecnología, herramienta o técnica capaz de resolver la crisis del software y producir una mejora radical en productividad, confiabilidad y simplicidad.

Esto se debe a que gran parte de la dificultad de desarrollar software es inherente a su propia naturaleza, y no solamente a las herramientas utilizadas.

1) Dificultades esenciales

Son las dificultades propias de construir software. No pueden eliminarse simplemente mejorando las herramientas.

Característica	Definición
Complejidad	El software es inherentemente complejo; al crecer su tamaño aumentan las relaciones entre sus componentes y las dificultades para comprenderlo y desarrollarlo.
Conformidad	El software debe adaptarse a los requerimientos y necesidades del cliente y su entorno.
Mutabilidad	El software está sometido a cambios constantes para responder a nuevas necesidades.
Invisibilidad	Al ser abstracto e intangible, resulta difícil visualizar y representar completamente su estructura.

2) Dificultades accidentales

Son dificultades asociadas a los medios y herramientas utilizados para construir software. Pueden reducirse mediante avances tecnológicos, pero resolverlas no elimina la complejidad esencial.

Ejemplos de mejoras:

- Lenguajes de programación de alto nivel.
- Sistemas de tiempo compartido.
- Entornos de programación integrados.
- Programación orientada a objetos, inteligencia artificial, programación automática y gráfica, herramientas de verificación y estaciones de trabajo más potentes.

Estas tecnologías facilitan el desarrollo, pero ninguna constituye por sí sola una «bala de plata».

3) Estrategias para afrontar las dificultades esenciales

- **Comprar antes que construir:** reutilizar soluciones existentes cuando satisfacen las necesidades, evitando desarrollar desde cero.
- **Prototipos y refinamiento de requerimientos:** comprender progresivamente las necesidades del cliente mediante iteraciones y feedback.
- **Desarrollo incremental:** construir y entregar funcionalidades pequeñas, utilizables y probadas, en lugar de esperar a completar todo el sistema.
- **Personas capacitadas:** contar con profesionales competentes y creativos, tanto en diseño como en liderazgo.

Unidad 2 – Gestión Lean-Agile de Productos de Software

Manifiesto Ágil/Filosofía Lean

Manifiesto Ágil

Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software.

Valores ágiles:



1. Individuos e interacciones sobre procesos y herramientas: se priorizan las personas, su motivación, habilidades y comunicación. Los procesos y herramientas deben adaptarse al equipo y facilitar su trabajo, no al revés.

2. Software funcionando sobre documentación extensiva: se prioriza entregar software funcional e incremental que permita obtener retroalimentación del cliente. Se genera la documentación necesaria, cuando aporte valor al producto o proyecto.

3. Colaboración con el cliente sobre negociación contractual: se busca involucrar activamente al cliente durante todo el desarrollo, trabajando en conjunto y obteniendo feedback frecuente. Los contratos siguen siendo necesarios, pero no deben dificultar la colaboración.

4. Responder a cambios sobre seguir un plan: se prioriza la adaptación frente a nuevas necesidades y requerimientos, en lugar de seguir rígidamente un plan inicial. Mediante la inspección y adaptación, el equipo ajusta la planificación para entregar el mayor valor posible al cliente.

Los 12 principios del Manifiesto Ágil:



Principio 1: Nuestra mayor prioridad es satisfacer al cliente: Un equipo ágil desarrolla y entrega funcionalidades en el orden especificado por el PO. Es ese rol quien determina la prioridad e importancia de cada funcionalidad, de manera que, en cada release del producto, se esté retornando al cliente el mayor valor posible. Esto está soportado por el hecho de trabajar con User Stories, las cuales plantean necesidades que tienen un valor significativo para los usuarios del software. De esta forma, el equipo de desarrollo se asegura de estar respondiendo a esas necesidades, al implementar User Stories, y no trabajar con tareas aisladas que no aportan valor para el cliente y/o los usuarios.

Principio 2: Aceptar que los requisitos cambien: Incluso en etapas tardías de desarrollo: El equipo ágil está abierto al cambio, al no seguir un plan estructurado que se confecciona en etapas tempranas, como plantea la metodología tradicional. Esto permite que el equipo sea flexible, y responda siempre ante las necesidades cambiantes de los clientes y/o usuarios.

Principio 4: Técnicos y no técnicos trabajando juntos durante todo el proyecto: (responsables de negocio, desarrolladores, diseñadores). Plantea la gran importancia de la participación del Product Owner, en conjunto con el equipo de desarrollo de software. Este rol, que generalmente es ocupado por una persona del “lado del cliente”, debe ser alguien capaz de tomar decisiones sobre el producto, y que además sea capaz de representar el interés y necesidades comunes de todos los usuarios a los que estará destinado el desarrollo del software.

Principio 6: El método más eficiente de comunicar información es con conversaciones cara a cara: Este principio tiene un sustento psicológico, el cual expresa que las palabras expresadas como sonido o texto, sólo representan un bajo porcentaje de la comunicación que se desea lograr hacia otra persona, siendo que la mayor parte está ligada a gestos, expresiones faciales, movimientos corporales, retroalimentación rápida, etc.

Principio 11: Las mejores arquitecturas, diseños y requerimientos surgen de

equipos autoorganizados: Este principio expresa que los equipos de desarrollo ágiles a diferencia de organizaciones jerárquicas tradicionales, (donde la dirección/gerencia o los jefes son quienes imparten órdenes, y todos deben acatarlas) deben tener integrantes del equipo proactivos, aportando soluciones, conceptos, herramientas, etc. que sean útiles al equipo. Este principio está sustentado por el empirismo, el cual establece que el conocimiento surge dentro del equipo de trabajo. El equipo toma las decisiones en el momento que ellos creen adecuados.

Filosofía Ágil

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación, que permiten una inspección y adaptación sobre los requerimientos cambiantes. Por otro lado, se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente en lugar de reuniones formales con documentos escritos.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema. Esta es una de las diferencias con el PUD, que, si bien utiliza un modelo de proceso iterativo - incremental, son procesos definidos.

La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales.

Pilares del empirismo:

- 1) Transparencia:** nos permite crecer como equipos y transformar el conocimiento que cada miembro del equipo tiene implícito en conocimiento explícito, es decir, que pase a formar parte del conocimiento de todo el equipo. Esto ayuda a que los procesos empíricos puedan fluir y generar esa experiencia necesaria para que el equipo evolucione.
 - Es la base para que se cumplan los otros 2 pilares
 - Es el que garantiza que el equipo gane experiencia en cada iteración/vuelta
 - La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.
- 2) Inspección:** puede realizarse para el producto, procesos, aspectos de personas, prácticas y mejoras continuas. Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados.
 - La inspección permite la adaptación. La inspección sin la adaptación se considera inútil.
- 3) Adaptación:** Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. Nos permite hacer ajustes en aspectos del proceso que se desvían fuera de los límites o si el producto es inaceptable. (deben hacerse lo antes posible para minimizar la desviación)

¿Qué es Ágil?

No es una metodología o un proceso, ni tampoco un framework completamente definido, sino que es una ideología con un objetivo definido de valores y principios que guían el desarrollo del producto. Esto se debe a que definen prácticas de cómo hay que organizarse, es decir, cómo debe organizarse el equipo.

Valores de los equipos ágiles

- Planificación continua, multi-nivel
- Facultados, auto-organizados, equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing Concurrente

¿Pero qué significa Ágil?

- Balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación:
 - o Los métodos ágiles son adaptables en lugar de predictivos.
 - o Los métodos ágiles son orientados a la gente en lugar de orientados al proceso

¿Por qué ir a ágil?

- Acelera entregas de productos de valor para el cliente.
- Permite adaptarse fácilmente a cambios de prioridades y requerimientos.
- Permite alinear los objetivos organizacionales y de TI.
- Reduce riesgos de proyecto.
- Permite una mejor visibilidad del proyecto.
- Mejora las relaciones en el equipo.
- Reduce costos de proyecto.

Algunos frameworks ágiles:

- SCRUM
- XP
- CRYSTAL
- SHAPEUP
- ATDD

Requerimientos en ambientes ágiles – User Stories

Fundamentación de los requerimientos ágiles

Los requerimientos ágiles se basan en 3 pilares:

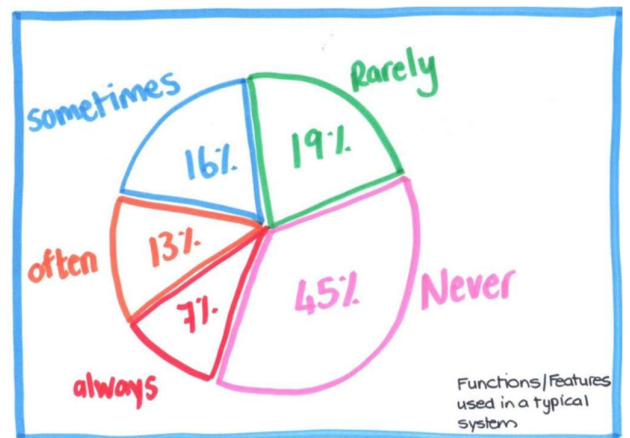
- **Usar el valor para construir el producto correcto:** Lo único importante es dejar contento al cliente con software funcionando que le sirva. Debemos enfocarnos en construir el producto correcto que le dé valor al negocio.
- **Determinar que es “sólo lo suficiente”:** los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Se tiene que empezar con lo mínimo necesario para generar realimentación y alimentar el empirismo y a partir de allí, se avanza continuamente.
- **Usar historias y modelos para mostrar que construir:** la parte de desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio ágil de “técnicos y no técnicos trabajando juntos”. Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo.

BRUF (Big Requirements Up Front o Grandes Requisitos Iniciales)

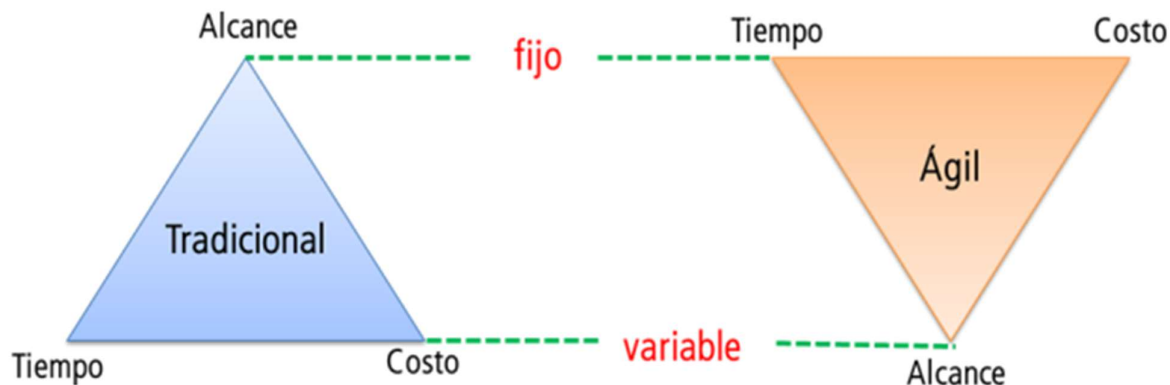
Consiste en intentar definir todos los requerimientos al inicio del proyecto. Su principal problema es que los usuarios no siempre conocen todas sus necesidades y estas pueden cambiar durante el desarrollo.

El enfoque ágil propone definir y priorizar los requerimientos progresivamente, desarrollando primero las funcionalidades de mayor valor para el cliente y adaptando las siguientes entregas a partir del feedback recibido.

La imagen demuestra que muchas funcionalidades de un sistema se utilizan poco o nunca, por lo que es fundamental priorizar aquellas que realmente aportan valor al usuario.

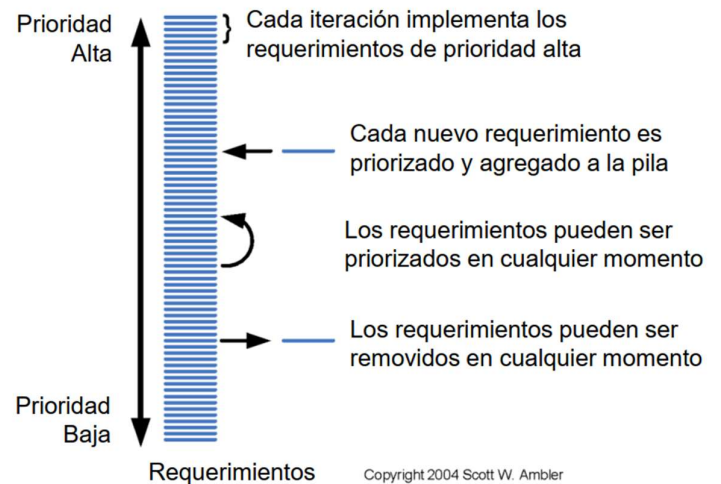


Tradicional vs Ágil



Product Backlog

La gestión ágil busca contrarrestar la situación planteada con anterioridad donde gran parte del software no se termina utilizando. El Product Backlog es una lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.



En contraposición con el enfoque tradicional, donde teníamos un cliente que no estaba dispuesto a comprometerse y dejaba la priorización de los requerimientos al equipo que hace el software, se terminaba construyendo un producto que no dejaba satisfecho al cliente. En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado.

Con esta situación se permite la compensación de no tener documentos de requerimientos formales escritos, detallando todos los puntos. Ágil ofrece una compensación al afirmar que la mejor comunicación es el cara a cara y un cliente participe en el equipo.

Requerimientos Just in Time

La gestión ágil de requerimientos sostiene los **requerimientos justo a tiempo (requerimientos emergentes)**, a diferencia del enfoque tradicional que plantea todos los requerimientos al principio.

Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. **El Product Backlog nunca está al 100%**, se va construyendo.

Tipos de requerimientos

La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

A nivel de la gestión ágil de requerimientos se trabaja con:

- Requerimiento de negocio
- Requerimiento de usuario

Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.

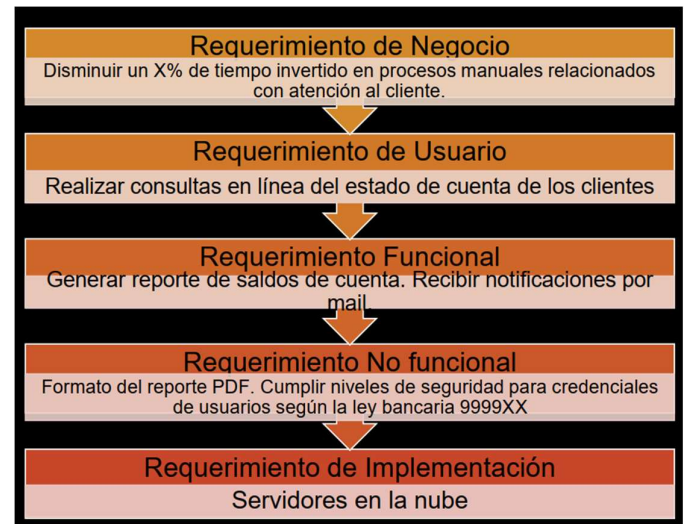
Al usuario le interesa el requerimiento de negocio. El valor de negocio es el requerimiento de negocio, pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio.

Las User Stories no sirven para especificar requerimientos de software, sirven para identificar requerimientos de usuario.

El foco de la gestión ágil de requerimientos está puesto sobre el usuario y sobre sus necesidades y por eso en enfoques como SCRUM quién debe trabajar para identificar las User Stories es el Product Owner. Ya que es él quien se concentra en las necesidades de usuarios que van a satisfacer las necesidades de negocio.

¿Qué debemos asumir?

- Los cambios son la única constante. Todo el tiempo va a haber cambios en los requerimientos.
- Stakeholders: no son todos los que están. Debemos saber quiénes son los involucrados y los involucrados son todos los que tiene algo que decir respecto del producto. El PO es el representante de todas estas personas y el que se comunica con todas ellas para luego ser él mismo el que se comunica con el equipo que hace el software.
- Siempre se cumple eso de que: "El usuario dice lo que quiere cuando recibe lo que pidió".
- No hay técnicas ni herramientas que sirvan para todos los casos. Debemos detectar para cada situación cuál es la herramienta más adecuada.
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de "valor".



Principios ágiles relacionados con requerimientos ágiles



1- LA PRIORIDAD ES SATISFACER AL CLIENTE A TRAVÉS DE RELEASES TEMPRANOS Y FRECUENTES (2 SEMANAS A UN MES)



2 -RECIBIR CAMBIOS DE REQUERIMIENTOS, AUN EN ETAPAS FINALES



4 - TÉCNICOS Y NO TÉCNICOS TRABAJANDO JUNTOS TODO EL PROYECTO



6 - EL MEDIO DE COMUNICACIÓN POR EXCELENCIA ES CARA A CARA



11 - LAS MEJORES ARQUITECTURAS, DISEÑOS Y REQUERIMIENTOS EMERGEN DE EQUIPOS AUTOORGANIZADOS

User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.

Funciona como un recordatorio para el equipo de desarrollo y el Product Owner de que tienen que conversar acerca de la necesidad en términos de negocio para la construcción de una característica del producto que aporte valor. En otras palabras, es un mecanismo para diferir una conversación.

Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Son escritas por el Product Owner.

Internamente, las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.

Se dice que son multipropósito, debido a que cumplen diferentes funciones en el desarrollo de un software:

- Representan una necesidad de usuario, de forma no detallada.
- Conforman una descripción del producto.
- Son utilizadas como ítems de planificación.
- Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
- Junto con el código ejecutable, permiten documentar el producto.

La problemática que motivó en la definición de esta técnica es la dificultad de comunicar lo que hará el sistema a todas las personas afectadas.

La definición de los requerimientos es lo que más afecta la calidad del sistema final y es lo más complejo de definir con detalle debido a la incertidumbre y cambio constante que sufren, que se da por el problema de comunicación. La dificultad se basa en qué se escribe y cuándo.

La elicitación tradicional no da buenos resultados, ya que todas las decisiones se toman al principio del proyecto cuando menos información se tiene. Por lo que se estableció que esta toma de decisiones se traslada a lo largo del proyecto, a medida que la incertidumbre baja y la información sube, buscando que se obtenga más a menor tiempo posible. Aquí surgen las User Stories.

Partes de una US (las 3 C)

- 1) Tarjeta (Card): es una descripción de la historia, utilizada para planificar y como recordatorio.
 - COMO <nombre del rol> QUIERO <actividad> PARA <valor de negocio>
 - **Nombre del rol**: representa quien está realizando la acción o quien recibe el valor de la actividad.
 - **Actividad**: representa la acción que realizará el sistema.
 - **Valor de negocio**: es la más importante y representa el por qué es necesaria la actividad. Esto justifica la razón de que el Product Owner escriba las US. Permite priorizar el desarrollo.
 - **Criterios de aceptación**: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad.
 - Se deben redactar independientes a la implementación (alto nivel de abstracción).
 - **Pruebas de aceptación**: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa. Las pruebas pueden agregarse o quitarse en cualquier momento.
- 2) Conversación (Conversation): Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US. El Product Owner es parte del equipo, por lo cual se espera responsabilidad, compromiso y confianza de su parte para hacerse cargo y llegar a un acuerdo común con el equipo sobre lo definido acerca de la funcionalidad. También es posible dejar la conversación persistente en grabaciones, minutas o softwares de gestión que posibiliten la conversación. De todas maneras, la conversación se materializa de forma distribuida en la tarjeta y en las pruebas de aceptación. En simples palabras, son discusiones acerca de la historia que sirven para desarrollar los detalles de la historia.
- 3) Confirmación (Confirmation): Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conforma a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Algo más de las US

- No son especificaciones detalladas de requerimientos (como los casos de uso)
- Son expresiones de intención, “es necesario que haga algo como esto...”
- No están detallados al principio del proyecto, elaborados evitando especificaciones anticipadas, demoras en el desarrollo, inventario de requerimientos y una definición limitada de la solución.
- Necesita poco o nulo mantenimiento y puede descartarse después de la implementación.
- Junto con el código, sirven de entrada a la documentación que se desarrolla incrementalmente después.

Ventajas

- Prioriza la comunicación verbal por sobre la escrita (recordar la conversación es la parte más importante).
- Son comprensibles tanto para el cliente/usuario (Product Owner) como para el equipo de desarrollo (recordar que se escriben en términos de negocio).
- Tienen el tamaño justo para permitir planificar.
- Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios.

Desventajas

- En proyectos grandes, es difícil entender la relación entre U.S (son independientes entre sí).
- No son apropiadas para equipos demasiado grandes (por el tema de priorizar la conversación verbal, por sobre la escrita).

Las US son multipropósito ya que son:

- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación

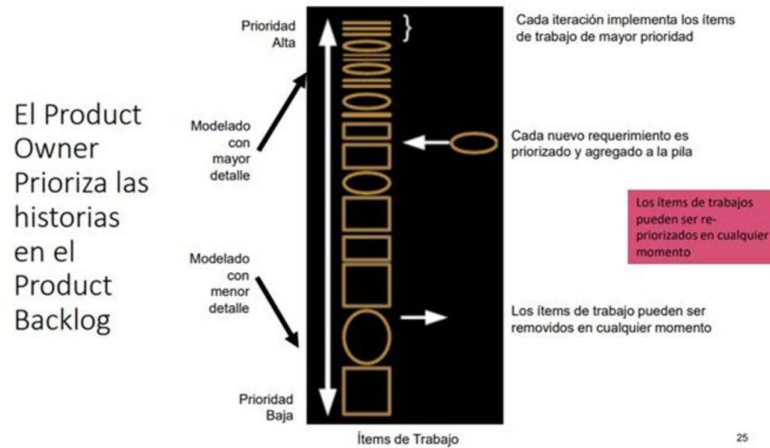
Product Backlog y las US

Es una pila de PBIs (Product Backlog Ítems) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (que son aquellas U.S. con granularidad fina), y las de menor prioridad (normalmente aquellas User de granularidad gruesa, sobre las cuales no se tiene mucho detalle ya que no se consideran indispensables) al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner, quien es el encargado de realizar el proceso de priorización.

En cualquier momento del proyecto se van agregando nuevas U.S. y actividades a realizar, según se vayan detectando nuevos requerimientos.

Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

Debido a estas dos situaciones (agregar y quitar ítems), nunca encontraremos el 100% de los requerimientos de usuario en el Product Backlog.



Porciones verticales

Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos.

Debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.

Relación con la arquitectura en capas

La User es una porción vertical, la cual abarca todas las capas de arquitectura: capa de datos, lógica de negocio y capa de presentación. Esto permite entregar una característica terminada al cliente.



Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes.

El tamaño de las U.S. se mide en Story Points.

Hay 3 aspectos que determinan el tamaño de una U.S.:

- **Complejidad:** hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas. Por ejemplo, algoritmos complejos, cálculos, gran cantidad de campos, muchas validaciones, demasiados filtros, etc.
- **Esfuerzo:** es el tiempo en horas de trabajo que requerirá la implementación
- **Incertidumbre**
 - o Técnica: cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
 - o De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Story Points

Unidad de medida relativa utilizada en ambientes ágiles para estimar el tamaño de una User Story, considerando su complejidad, esfuerzo e incertidumbre.

Se estiman mediante la comparación entre historias y no representan una medida de tiempo. Generalmente se utiliza la serie de Fibonacci (1, 2, 3, 5, 8, 13...). Permiten estimar el trabajo y facilitar la planificación de las iteraciones.

Los Story Points indican el tamaño relativo de una User Story, no cuánto tiempo exacto llevará desarrollarla.

Modelado de roles

Se utilizan las tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario, respecto al producto y en relación con su perfil genérico.

Técnicas adicionales:

1. **Personas:** esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serían los objetos.
 - o Mario, 28 años, es ingeniero informático y busca cambiar de trabajo. Utiliza frecuentemente aplicaciones de empleo desde su celular y necesita filtrar ofertas por modalidad remota, salario y ubicación para encontrar oportunidades acordes a su perfil.
2. **Personajes extremos:** que personas en términos extremos podrían llegar a utilizar el software.
 - o Diseño de un PDA (Asistente Digital Personal) para:
 - El Papa
 - Una mujer con muchos novios
 - Un traficante de drogas
 - o La mujer y el traficante desearán mantener agendas separadas en caso de que la vea la policía o un novio. El Papa probablemente tenga menos necesidad de discreción, pero querrá un tamaño de letra más grande.
3. **Proxies (Usuarios Representantes):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.
 - o Gerentes de desarrollo
 - o Alguien del grupo de marketing
 - o Vendedores
 - o Expertos del dominio
 - o Clientes
 - o Capacitadores

Técnica	Característica principal
Personas	Perfil ficticio y realista que representa a un tipo de usuario.
Personajes extremos	Perfiles inusuales que permiten descubrir necesidades no contempladas.
Usuarios representantes (Proxies)	Personas reales que actúan como intermediarios entre los usuarios y el equipo de desarrollo.

Criterios de aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que este debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Lo fundamental es que esté escrito en términos objetivos y concretos.

El criterio de aceptación es información concreta que tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no. Debemos saber si el PO nos va a aceptar esta característica implementada o no, porque la va a aceptar si respeta los criterios de aceptación.

- Definen límites para una User Story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos).
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US.

¿Cuáles son los criterios de aceptación buenos?

- Definen una intención, no una solución.
 - o Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- Son independientes de la implementación.
- Relativamente de alto nivel, no es necesario que se escriba cada detalle.

¿Y los detalles, a dónde van?

- El encabezado de la columna se nombra "Saldo"
- El formato del saldo es 999.999.999,99
- Debería usarse una lista desplegable en lugar de un Checkbox.

Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:

- Documentación interna de los equipos
- Pruebas de aceptación automatizadas

Pruebas de aceptación

Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que **no es posible escribir todas las posibles pruebas de aceptación.**

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; suelen usarse para completar detalles de la US. Es importante mencionar que las mismas deben escribirse antes que la programación empiece, y que se recomienda que las escriba el cliente.

Proceso de 2 pasos:

- 1) Identificarlas al dorso de la US.
- 2) Diseñar las pruebas completas

DoR (Definition of Ready) – Definición de Listo

Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model.

DoD (Definition of Done) – Definición de Hecho

Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. Se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.

En la práctica consta de un Checklist de condiciones que deben cumplirse.

INVEST Model

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

- **Independiente:** la User es calendarizable e implementable en cualquier orden. Esto implica que no existe una dependencia con otra User para desarrollarla o para mostrarla al Product Owner. El PO tiene la libertad de priorizarlas en el Product Backlog como él desee.
- **Negociable:** Las US no son contratos estrictos, sino más bien pequeñas descripciones de funcionalidades donde se describen las necesidades de los usuarios, y los detalles se dejan para negociar en conversaciones futuras entre clientes y el equipo de desarrollo. Esto implica que no deben detallarse todos los aspectos de forma exhaustiva, sino más bien, servir a modo de recordatorio para negociar aquellos aspectos en un futuro cuando se deba implementar. La US debe estar escrita en términos de qué necesita el usuario y no de cómo lo vamos a implementar.
- **Valuable:** Las U.S. deben aportar valor de negocio a quiénes estará destinado el producto, no para el equipo de desarrollo. Esto implica dejar de lado en US, por ejemplo, las tecnologías utilizadas para el desarrollo del producto.

- **Estimable:** la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo. Sin poder estimarla no es posible determinar si se puede finalizar en una iteración por lo tanto no puede ser construida (No puedo saber cuánto tardaría). Hay tres razones por las cuales una US puede no ser estimable:
 - 1) El equipo tiene incertidumbre acerca del dominio. (Deberán discutir con el Product Owner para esclarecer todas las dudas.
 - 2) El equipo tiene incertidumbre técnica, es decir, sobre el dominio de la solución. (Deberá destinarse uno o más miembros del equipo a uno o más Spikes)
 - 3) La US es demasiado grande. (Deberá dividirse en US más pequeñas.)
- **Small (Pequeña):** la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración. Se debe comenzar y terminar una característica en una iteración. El tamaño es relativo, ya que depende del equipo, de la experiencia que tiene, de su capacidad de trabajo y también depende de la duración de la iteración. Ágil aplica gestión binaria, es decir, si se puede completar la US en la iteración, es pequeña. No aplica porcentaje de avances.
- **Testeable:** la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada (se muestra que está hecha y demostrable).

Niveles de abstracción

1. **User Story:** cumple con el criterio de Listo (DoR) para ingresar a una iteración.
2. **Épica:** es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User. Una épica se suele identificar también cuando no cumple con el modelo de calidad de INVEST. La épica debe ser dividida en US más pequeñas. Sirven para aquellos requerimientos que no son prioridades en las primeras iteraciones, y basta con tener sólo una aproximación general de las funcionalidades requeridas. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.
3. **Temas:** son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.
(Épicas y Temas) permiten reducir los esfuerzos del equipo a la hora de estimar.

Spikes

Son un tipo especial de US que **se producen por la incertidumbre** que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US. Es una característica más del producto a la cual el equipo le tiene que dedicar tiempo para:

- 1) Familiarizarse con una nueva tecnología o dominio.
- 2) Investigar y prototipar para ganar confianza frente a:
 - a. Riesgos tecnológicos.
 - b. Riesgos funcionales, donde no está claro cómo debe reaccionar el sistema para satisfacer las necesidades del usuario.

Se usan para quitar el riesgo e incertidumbre de una US y otra faceta del proyecto. Se clasifican en técnicas y funcionales y pueden utilizarse para:

- 1) Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
- 2) Analizar el comportamiento de una historia compleja y poder dividirla en piezas manejables.
- 3) Ganar confianza frente a riesgos tecnológicos, investigando o prototipando.
- 4) Ganar confianza frente a riesgos funcionales, donde no está claro cómo el sistema debe resolver la interacción con el usuario para alcanzar el beneficio esperado.

Spikes técnicas

- Utilizadas para investigar enfoques técnicos en el dominio de la solución.
 - o Evaluar performance potencial
 - o Decisión hacer o comprar
 - o Evaluar la implementación de cierta tecnología.
- Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.

Spikes funcionales

- Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema.
- Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados

Combinación de ambas

Algunas User Stories requieren de ambos tipos de spikes. Por ejemplo:

Como un cliente, quiero ver mi uso diario de energía en un histograma, para poder comprender rápidamente mi consumo de energía pasado, presente y proyectado.

En este caso un equipo puede crear dos spikes:

- **Spike Técnico:** Investigar cuanto tiempo requiere actualizar un display de un cliente al uso actual, determinando requerimientos de comunicación, ancho de banda y si los datos se actualizan en formato push o pull.
- **Spike Funcional:** Crear un prototipo de histograma en el portal web y obtener la retroalimentación de algunos usuarios respecto del tamaño, el estilo de la presentación y los atributos gráficos.

Lineamientos para Spikes

- Estimables, demostrables, y aceptables.
- La excepción, no la regla.
 - o Toda historia tiene incertidumbre y riesgos.
 - o El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
 - o Los spikes deben dejarse para incógnitas más críticas y grandes.
 - o Utilizar spikes como última opción.
- Implementar la spike en una iteración separada de las historias resultantes.

- Generalmente se realizan en iteraciones separadas: primero la Spike para resolver la incertidumbre y luego las historias resultantes para implementar la funcionalidad. Excepcionalmente, si la Spike es pequeña y sencilla, puede realizarse junto con la User Story en una misma iteración.
 - Por ejemplo, si el equipo solamente necesita investigar durante unas horas cómo utilizar una función de una biblioteca conocida, podría investigar e implementar la funcionalidad dentro de la misma iteración. Eso sí: debe existir tiempo suficiente para completar ambas actividades.



DIFERIR EL ANÁLISIS
DETALLADO TAN TARDE
COMO SEA POSIBLE, LO QUE
ES JUSTO ANTES DE QUE EL
TRABAJO COMIENCE.



HASTA ENTONCES, SE
CAPTURAN
REQUERIMIENTOS EN LA
FORMA DE "USER STORIES".



LAS USER STORIES NO SON
REQUERIMIENTOS DE
SOFTWARE, NO NECESITAN
SER DESCRIPCIONES
EXHAUSTIVAS DE LA
FUNCIONALIDAD DEL
SISTEMA.

Estimaciones de software

Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto. Se relaciona con los objetivos del negocio, compromisos y control.

Según McConell una estimación en el contexto del software es una predicción de cuánto tiempo durará o costará un proyecto. Es considerada una de las actividades más complejas en el desarrollo de software.

Las estimaciones no son planes ni compromisos, sino que sirven de base para planificar, pero el plan final no tiene por qué coincidir con lo estimado.

- Estimar = Predecir
- A través de una técnica recaudar información para predecir acerca del futuro
- No da la planificación del proyecto, da una base para planear mi proyecto
 - El compromiso es el plan y para formular el plan yo voy a tomar como base el resultado de la estimación
- Hay un cierto nivel de incertidumbre (al principio es mayor)
- NO ES ESTÁTICO → está vivo todo el tiempo, no ocurre solo una vez
- A medida que el tiempo pasa, la estimación se hace más precisa
 - Cuando el proyecto está terminando, la estimación ya no nos sirve para nada

Tips respecto de las estimaciones

- Si las estimaciones se utilizan como compromisos son muy peligrosas y perjudiciales para cualquier organización.
- Lo más beneficioso en las estimaciones es el “proceso de hacerlas”.
- La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no.
- La estimación puede servir como una gran protección para el equipo.

Errores en las estimaciones

1. Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
2. Demasiado caos en el proyecto (mal definido el proyecto).
3. Imprecisión generada por el proceso de estimación.
4. Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones, etc.
5. Se olvidan del detalle que planear no es estimar y estimar no es planear, sino que las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
6. A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.
7. Las estimaciones no son compromisos.

¿Por qué estimamos?

Existen dos razones principales por las cuales se llevan adelante las estimaciones:

- Para predecir completitud
- Para administrar riesgos

Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el **trabajo** que habrá de realizarse, los **recursos** que se requerirán y el **tiempo** que transcurrirá desde el principio hasta el final. Las estimaciones requieren:

Siempre se desea saber muy al inicio del proyecto cuál será el costo, el tiempo que llevará, entre otros aspectos y como aún no se conoce con precisión cuál será el alcance del producto resulta complejo predecir con exactitud estos aspectos, es por ello que a medida que avanza el proyecto, las estimaciones son más certeras, dado que se cuenta con mayor información sobre la cual realizar las estimaciones. (Es por ello que se suele decir que típicamente la primera estimación difiere hasta un 400%).

Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más, esto se debe a 3 aspectos:

- 1) Al inicio del proyecto tenemos poca información y demasiada incertidumbre, lo que aumenta el riesgo de generar estimaciones erróneas.
- 2) Las estimaciones son de naturaleza optimista, por eso las estimaciones deben ser recalculadas conforme se avanza con el proyecto.
- 3) Se suele decir: “el que no sabe estimar, estima muchas veces, y el que no sabe planificar, planifica muchas veces”.

Métodos utilizados para estimar

1) **Basados en la experiencia:**

- a. Datos históricos: consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones, con esto se busca una alternativa en la cual el conocimiento de cada individuo se transfiere a la organización y no estamos atados a una única persona. Se deben utilizar para no tener problemas entre desarrolladores y clientes. Esto no va acorde a la gestión ágil, que dice que la experiencia no se puede extrapolar a otros equipos.
 - Es difícil poder comparar esos datos históricos porque ningún proyecto es igual a otro
- b. Juicio experto:
 - i. Juicio de experto puro: un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. (el problema es que, si el experto se va de la organización, esta pierde su capacidad de estimación). Se puede utilizar el “método de optimista, pesimista, habitual”, en el que se le pregunta al experto sobre 3 valores distintos y se aplica una fórmula para calcular la estimación.

Problemas:

- ¿Cómo defino al experto?
- ¿Cómo valido los conocimientos que tiene este experto?
- ¿Qué pasa si el experto desaparece??

Es el método más utilizado en la práctica

- ii. Juicio Wideband Delphi: es similar al anterior pero grupal, un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración. Hay un coordinador que va a reunir las estimaciones individuales de cada uno de los expertos, se opina sobre las estimaciones ajenas de manera anónima, se vuelve a discutir, revisan, las vuelven a enviar al coordinador y se repite hasta que todos estén de acuerdo de forma razonable. Este método es tomado como base para el **Poker**

Planning.

- 2) **Analogía**: Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares. Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro. Es decir, consiste en tomar a un proyecto e ir comparando factores, por ejemplo:
 - **Tamaño**: es menor o mayor al proyecto anterior.
 - **Complejidad**: más complejo de lo usual.
 - **Usuarios**: si hay más usuarios habrá más complicaciones, como la concurrencia o los datos a almacenar.

Estimaciones tradicionales

- Estimamos el tamaño del software, el esfuerzo, el tiempo y los recursos o costo que vamos a tener
- Orden de estimación:
 - 1) Tamaño
 - 2) Esfuerzo (horas lineales que me lleva a hacer una actividad)
 - 3) Calendario
 - 4) Costos, recursos, etc.
- Somos extremadamente optimistas, nos cuesta pensar en términos de lo que va a salir mal
- No considera imprevistos o trabajo adicional como deuda técnica, participación en otros proyectos, licencias, días de enfermedad

Problemas de estimaciones tradicionales:

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

Estimaciones en ambientes ágiles:

- Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra.
- Según la filosofía ágil quien estima es quien debe hacer el trabajo. Es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones).
- La estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, volviendo más precisa la estimación (a diferencia de las metodologías tradicionales, donde se estima individualmente y se depende sólo de ese miembro).
- Las estimaciones se tienden a confundir con compromisos o planificación. Uno planifica basándose en la estimación, pero no son lo mismo.
- No es una instancia única, ya que a medida que se avance el proyecto, más información vamos a tener y más eficiente van a ser nuestras estimaciones.
- El gran beneficio del proceso de estimación es justamente "hacerlas". Al estimar, se enriquece el trabajo del equipo, lo cual mejorará futuros procesos de estimaciones. Además, este proceso ayuda a determinar la factibilidad del trabajo planificado en etapas tempranas.
- Tamaño, tiempo, costo → medidas absolutas (son lineales)
- El tiempo de estimación se define y asigna con anterioridad para no utilizar más esfuerzo del necesario.
- Para las estimaciones se deben tener en cuenta tres elementos a la hora de estimar: complejidad, esfuerzo e incertidumbre.

Consideraciones (Estimación relativa)

- Son medidas relativas no absolutas.
- No es una medida basada en el tiempo.
- Las personas no saben estimar en términos absolutos, somos buenos comparando.
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- Se emplea mejor el tiempo de análisis de las Stories.

¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de las US usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes, la serie de Fibonacci.

Al ser estimaciones de tamaños relativos, (es decir, se estima realizando comparaciones de los tamaños de las US) lo importante en este proceso no es el valor que se asigna a cada US en particular, sino más bien que las estimaciones sean consistentes en el conjunto. Es decir, para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, pero lo que importa es que, en un mismo equipo, una US de 5 puntos, siempre sea más chica que una de 8.

Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija. Por esto, es que no se recomienda estimar el tamaño de las US en horas, para no confundir con el esfuerzo.

Tamaño vs Esfuerzo

Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación, conlleva horas lineales (horas únicamente de trabajo, sin tener en cuenta almuerzo, baño, leer emails, etc. Para una jornada de trabajo de 8 horas, usualmente solo 5.5 o 6 son lineales). Por otro lado, no se debe confundir esfuerzo con calendario, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

Velocidad

- Es una medida (métrica) del progreso de un equipo. Se calcula sumando el número de story points (asignados a cada US) que el equipo completa durante la iteración.
- Se cuentan los story points de las Users Storys que están completas, no parcialmente completas.
- La Velocidad corrige los errores de estimación.

Tamaño

Dado que el tamaño es lo que se compara, lo definimos como la medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para hacer o completar una Feature/US. También el tamaño es función de la incertidumbre de dicha US, definida la incertidumbre como la falta de información.

El tamaño no es lo mismo que el esfuerzo (el “quien” define esta diferencia), ya que este último indica las horas humano lineal requeridas para el desarrollo de la US. Por lo tanto, una US de mismo tamaño puede implicar esfuerzos diferentes en función de la persona que se esté evaluando, debido a que el esfuerzo varía dependiendo de las habilidades, conocimientos, experiencia, familiaridad con el dominio, etc.

Formas de estimar el tamaño:

Existen diferentes escalas para estimar el tamaño y siempre son por comparación, entre ellas:

- 1) Número del 1 al 10.
- 2) Talles de remeras: S, M, L: suele ser utilizado como escala de estimación para los ítems del Product Backlog.
- 3) Serie exponencial de base 2: 2, 4, 8, 16, 32, 64, etc.
- 4) Serie de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

Una vez definida la escala, no se cambia. En caso de cambiarla, también se debe cambiar el patrón de medición.

Story Point

Es una unidad de estimación RELATIVA que especifica la complejidad, incertidumbre y esfuerzo propio del equipo respecto a una US en términos relativos respecto a otra US. Es lo que le da la idea del “peso” de cada US, decide cuán grande/compleja es (y a partir de ese peso es la comparación que vamos a hacer entre una historia y la otra). Por lo general esa complejidad tiende a incrementarse exponencialmente.

- **Complejidad:** que tan compleja es implementar esa US. Relacionada a la cantidad de partes y relaciones entre las mismas posee la US.
- **Esfuerzo:** es importante que estime la persona que va a hacer el trabajo, ya que el esfuerzo es distinto entre dos personas. Son las horas persona lineales que se requieren para terminar la US.
- **Incertidumbre:** es cuánta información falta para ser más adecuados en la estimación. Existe una incertidumbre técnica que es más sobre si se sabe cómo llevar a cabo lo explicitado con respecto a la tecnología a implementar y ese tipo de cosas. Y tenemos incertidumbre del negocio, si entendemos que es lo que el cliente pide y cómo llevarlo a cabo.

Reestimación

Estimar nuevamente una US es aconsejable sólo cuando la opinión acerca del tamaño relativo de una US ha cambiado, debido a que en el proceso de estimación se han detectado otras U.S. con tamaños más grandes o pequeños. (para mantener la consistencia de tamaños a la hora de estimar). No se debería reestimar porque no se detecta un progreso como el esperado (velocidad del equipo), sino que se debe dejar que la velocidad a lo largo de sucesivas iteraciones equilibre esas inexactitudes.

Poker Planning o Poker Estimation

Es una técnica de estimación en ambientes ágiles publicada por Mike Cohn. Resulta de la conjunción de diferentes métodos como el de juicio experto, analogía y desagregación, basado en que 4 ojos ven más que 2. En contraposición al método de juicio experto tradicional, asume que las personas que desarrollan el producto deben ser aquellas que estimen. En otras palabras, el equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. Un detalle es que el Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

- Es la base de Wideband Delphi
- Se elige una US canónica (lo suficientemente pequeña para que entre todos podamos asignarle el valor 1), que nos va a servir de base para comparar
 - o A partir de ahí vamos a hacer rondas (como Delphi) utilizando las cartas

Escala (Serie de Fibonacci)

- 1) **Valor 1 o 1/2:** son los valores más bajos y se aplican a una funcionalidad pequeña.
- 2) **Valor 2- 3:** funcionalidad pequeña a mediana.
- 3) **Valor 5:** funcionalidad media.
- 4) **Valor 8: funcionalidad** grande, donde nos preguntamos si es posible dividir.
- 5) **Valor 13 o más:** necesariamente hay que dividir ya que no ingresa en el sprint.

Procedimiento

- 1) Se define una lista de actividades, módulos, casos de uso, Users stories, etc.
- 2) Se acuerda cuál de las anteriores es la que tiene **menor grado de incertidumbre** y **se la define como canónica** asignándole 1 o 2 story point (es la más simple). Esta User es utilizada como referencia para comparar.
- 3) Cada miembro elige una carta con el valor de la estimación del tamaño de la y se espera a que todos decidan.
- 4) Utilizando **juicio experto**, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones. Luego el resto del medio aportan sus opiniones. Siempre la escucha es activa, para poder aprender.
- 5) En caso de que en la primera vuelta existiesen diferencias considerables, se realiza nuevamente la ejecución del paso 4.
- 6) Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

Ventajas

- Permite opiniones multidisciplinarias de todo el equipo, lo cual enriquece la estimación.
- Al generar debates y discusiones, permite lograr una mejor comprensión del dominio en los miembros del equipo.
- Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que estimar de forma grupal.
- Es una actividad que resulta divertida para el equipo.

Gestión de Productos de Software

Un producto de Software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos.

Cuando creamos un producto debemos enfocarnos en aquello que es de valor para el cliente, sabiendo que el 7% de las características del software son siempre usadas. Esto implica, que la mayor parte del valor agregado está concentrado en ese 7%. Alrededor del 80% de las características del software que se desarrolla no se utilizan. El desafío está en encontrar ese 7%.

Las organizaciones que usan el software se enfocan en la parte superficial del software, aquello que consideran significativo y placentero. Usualmente no se fijan si el producto es funcional o confiable. Esa debe ser la base, pero el producto debe proveer una experiencia que cumpla con todas las expectativas del usuario.

Creamos productos para:

- Satisfacer a los clientes.
- Tener muchos usuarios logueados. Una métrica que se utiliza para saber que tan aceptado es un producto, es por la cantidad de usuarios que lo instalan.
- Para obtener mucho dinero.
- Realizar una gran visión, cambiar el mundo.

Mínimo Producto Viable:

Es la versión más simple de un producto que permite probarlo con clientes reales y aprender de su comportamiento con el menor esfuerzo posible.

Su objetivo principal es validar si el producto satisface una necesidad real antes de invertir más tiempo y recursos en desarrollarlo completamente.

Idea clave: aprender observando lo que los clientes realmente hacen con el producto, en lugar de basarse únicamente en lo que dicen que harían.

Ejemplo: antes de desarrollar una aplicación completa de venta de ropa, se lanza una versión básica que permite consultar productos y realizar compras. Se observa si los clientes la utilizan y qué necesitan mejorar o incorporar.

Características

- 1) Tiene el valor suficiente para que las personas estén dispuestas a usarlo o comprarlo.
- 2) Demuestra beneficios a futuro para retener a los usuarios.
- 3) Proporciona un ciclo de retroalimentación para guiar el desarrollo a futuro.

Los errores más comunes son:

- 1) Confundirlo con MMF o MMP. La MMF (Característica Comercializable Mínima) o el MMP (Producto Comercializable Mínimo) a diferencia del MVP se enfocan en **ganar**, en cambio un MVP se enfoca en el aprendizaje
- 2) Enfatizar la parte de mínima y dejar de lado la parte de viable. Si el producto no tiene la calidad suficiente, no será posible obtener la retroalimentación del cliente.
- 3) Hacer del MVP un producto final, es decir, entregar lo que consideran un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto.

Preparar un MVP

- 1) Encontrar un nicho de mercado.
- 2) Roadmap: definición de muy alto nivel de qué características del producto se podrían tener a cierta altura del año. Esto **sirve para los inversores**.
- 3) Investigar la competencia: ver hasta dónde llega, qué hace, qué fortalezas tiene.
- 4) Pre-vender el MVP.
- 5) Testear las distintas suposiciones.
- 6) Asegurarse de que el MVP esté resolviendo el problema correcto.
- 7) Enfocarse solo en las funcionalidades principales.
 - Se construye en fetas verticales

MVF (Minimum Viable Feature) – Característica Mínima Viable

Es la versión más simple de una funcionalidad que permite probarla con usuarios reales y aprender si resulta útil, utilizando el menor esfuerzo posible.

Su objetivo es validar si la funcionalidad tiene valor y es utilizada antes de invertir más recursos en desarrollarla completamente. Es como un MVP, pero aplicado a una funcionalidad específica.

Valido la característica que haga la diferencia entre el producto que yo ofrezco y otro que esté disponible en el mercado

MMF (Minimum Marketable Feature) – Característica Mínima Comercializable

Es la funcionalidad más pequeña que puede desarrollarse y lanzarse al mercado, aportando valor tanto a los usuarios como a la organización.

Su objetivo es entregar valor rápidamente, sin esperar a desarrollar todas las funcionalidades previstas. (Ya no estoy validando una hipótesis, estoy en un release)

Concepto	¿Qué es?	Objetivo principal
MVP (Mínimo Producto Viable)	Versión más simple de un producto nuevo.	Validar si el producto satisface una necesidad real y aprender de los clientes con el menor esfuerzo.
MVF (Característica Mínima Viable)	Versión más simple de una funcionalidad específica.	Probar con usuarios reales si una funcionalidad resulta útil antes de desarrollarla completamente.
MMF (Característica Mínima Comercializable)	Funcionalidad más pequeña que puede lanzarse al mercado y aportar valor.	Entregar valor rápidamente a los usuarios y a la organización.

Ejemplo: aplicación de venta de ropa

Queremos incorporar una funcionalidad que recomiende prendas según los gustos del cliente.

MVF — Validar una hipótesis

Todavía no sabemos si los clientes utilizarán las recomendaciones.

Desarrollamos una versión muy sencilla y la habilitamos para 20 usuarios. Observamos si utilizan las recomendaciones y si les resultan útiles.

Si funciona, continuamos desarrollándola. Si no funciona, podemos modificarla o descartarla.

Objetivo: aprender con el menor esfuerzo posible.

MMF — Entregar valor

Ya tenemos evidencia de que las recomendaciones son útiles para los clientes.

Desarrollamos una funcionalidad pequeña pero completa: mostrar cinco prendas recomendadas según las compras anteriores del usuario.

La lanzamos para que los clientes puedan utilizarla y aporte valor al negocio.

Objetivo: entregar valor rápidamente.

Comparación para recordar

Aspecto	MVF	MMF
¿Existe incertidumbre sobre su valor?	Sí	Menor; tenemos evidencia de valor
¿Qué buscamos?	Aprender y validar	Entregar valor
¿Con quiénes se prueba o utiliza?	Grupo reducido de usuarios reales	Usuarios a los que se dirige la funcionalidad
¿Qué hacemos con los resultados?	Decidir si continuar, modificar o descartar	Continuar entregando funcionalidades de valor

La MVF busca descubrir si una funcionalidad aporta valor; la MMF busca entregar ese valor mediante una funcionalidad pequeña y comercializable.

	MVP (Viable)	MMP (Comercializable)	MLP (Adorable)
Objetivo Principal	Validar concepto de forma económica	Atender a base amplia con funcionalidad clave	Crear conexión emocional e interacción profunda
Enfoque de Diseño	Investigar y empatizar	Construir y refinar / Código y testing	Diseñar para el deleite / Impacto emocional
Beneficio Principal	Informar el alcance del producto	Asegurar la calidad y salida al mercado	Aumentar la lealtad y el amor del usuario
Ejemplo Típico	Entrevistas y videos	Lanzamiento Beta	App con animaciones UI divertidas y personalizadas

SCRUM

- Es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos. En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:

- 1) Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.
 - 2) El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
 - 3) El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
 - 4) Repetir
- Scrum se basa en el empirismo y el pensamiento Lean.
- El empirismo afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa.
 - El pensamiento Lean reduce los desperdicios y se centra en lo esencial.
 - Scrum emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo.
 - Scrum involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.

Principios de SCRUM

- 1) Control de proceso empírico, experiencia, aprender de la experiencia
- 2) Autoorganización del equipo, nadie indica qué hacer
- 3) Colaboración entre todas las partes
- 4) Priorización basada en valor → en el product backlog se pone + arriba lo que tiene + valor
- 5) Time-boxing
- 6) Desarrollo interactivo → funciona con ciclo de vida interactivo e incremental

Pilares de SCRUM

Transparencia: El proceso y el trabajo emergentes deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo. Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo. La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

Inspección: Los artefactos de Scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y minuciosamente para detectar varianzas o problemas potencialmente indeseables. La inspección permite la adaptación. La inspección sin adaptación se considera inútil. Los eventos de Scrum están diseñados para provocar cambios.

Adaptación: Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar la desviación adicional. La adaptación se vuelve más difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para

autogestionarse. Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

Empirismo y el ciclo OODA

- Nuevo enfoque
- Es un ciclo continuo que permite aprender de la realidad, tomar decisiones y adaptarse rápidamente a los cambios.
- **Observar** → recopilar información y evidencias sobre lo que está sucediendo, sin ocultar problemas.
- **Orientar** → interpretar la información obtenida para comprender la situación y sus causas.
- **Decidir** → elegir qué acciones o experimentos realizar para responder a la situación.
- **Actuar** → ejecutar las decisiones tomadas y observar sus resultados para comenzar nuevamente el ciclo.

Los 5 valores de SCRUM

- 1) **Compromiso:** el equipo se compromete a alcanzar sus objetivos y a apoyarse mutuamente para lograrlos.
- 2) **Enfoque:** el equipo concentra sus esfuerzos en el trabajo del Sprint para avanzar hacia sus objetivos.
- 3) **Apertura:** el equipo y los interesados son transparentes respecto al trabajo, los avances y las dificultades que surgen.
- 4) **Respeto:** los integrantes se respetan mutuamente, reconociendo las capacidades, conocimientos e independencia de cada persona.
- 5) **Coraje:** los integrantes tienen el valor de afrontar problemas complejos, tomar decisiones difíciles y hacer lo correcto.

El ecosistema extendido: Stakeholders y Supporters

- Stakeholders (interesados): Clientes, usuarios, decisores, e incluso entidades no humanas (la ley, IA)
 - o Rol → proveen feedback, requisitos y validan el valor para construir SW
 - o Es un rol externo
 - o Ya existían, ahora se plantean como rol
- Supporters (Agentes de cambio) → líderes y gerentes que apoyan activamente la adopción de SCRUM
 - o Rol → eliminan barreras organizacionales y proveen recursos (humanos, tecnológicos, etc)
 - o Si un manager no empodera al equipo ni remueve impedimentos no es un supporter
- Product owner NO es un supporter

Roles en la Guía Expandida

Rol	Naturaleza	Responsabilidad Principal
Product Owner	Humano	Maximizar el valor a largo plazo; gestionar el Product Backlog y la estrategia del producto.
Product Developer	Humano o Automatizado	Crear incrementos utilizables; adherirse a la Definición de Salida Terminada; autogestionar el plan del Sprint.
Scrum Master	Humano	Agente de cambio; guía la efectividad del equipo y la organización en la adopción de Scrum.
Stakeholder	Humano/Entidad/IA	Entidades interesadas o afectadas por el producto (clientes, usuarios, patrocinadores, legisladores).
Supporter	Stakeholder específico	Coalición poderosa que elimina impedimentos organizacionales y fomenta un clima para la autogestión.

Scrum Team

- **Composición:** está formado por un Product Owner, un Scrum Master y los Developers (Desarrolladores).
- **Sin jerarquías:** no existen subequipos ni jerarquías internas. Es una unidad cohesionada enfocada en el Objetivo del Producto.
- **Multifuncional:** sus miembros poseen todas las habilidades necesarias para generar valor en cada Sprint.
- **Autogestionado:** el equipo decide internamente quién hace qué, cuándo y cómo.
- **Tamaño reducido:** generalmente está compuesto por 10 personas o menos, favoreciendo la comunicación y productividad.
- **Responsabilidad integral:** se encarga de todas las actividades relacionadas con el producto (investigación, desarrollo, pruebas, mantenimiento, operación, etc.).
- **Ritmo sostenible:** trabaja mediante Sprints, manteniendo un ritmo de trabajo constante.
- **Responsabilidad compartida:** todo el Scrum Team es responsable de crear un incremento valioso y útil en cada Sprint.

Importante: si el equipo crece demasiado, puede dividirse en varios Scrum Teams que trabajen sobre el mismo producto, compartiendo el Objetivo del Producto, Product Backlog y Product Owner.

Roles de SCRUM

Desarrolladores

Los desarrolladores son las personas del equipo Scrum que se comprometen a crear cualquier aspecto de un Incremento útil (funcional) en cada Sprint. Responsables de:

- Crear un plan para el Sprint, el Sprint Backlog;
- Inculcar la calidad adhiriéndose a una definición de Hecho
- Adaptar su plan cada día hacia el Objetivo Sprint
- Responsabilizarse mutuamente como profesionales.

Propietario del producto (Product Owner)

Es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. También es responsable de la gestión eficaz de la pila del producto (Product Backlog), que incluye:

- Desarrollar y comunicar explícitamente el Objetivo del Producto;
- Creación y comunicación clara de elementos de trabajo pendiente del producto;
- Pedido de artículos de trabajo pendiente del producto;
- Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido.

Scrum master

El Scrum Master es responsable de establecer Scrum tal como se define en la Guía de Scrum. Lo consigue ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo como en toda la organización. Responsable de:

- Capacitar a los miembros del equipo en autogestión y multifuncionalidad;
- Ayudar al equipo de Scrum a centrarse en la creación de incrementos de alto valor que cumplan con la definición de hecho;
- Promover la eliminación de los impedimentos para el progreso del equipo Scrum;
- Asegurar de que todos los eventos de Scrum se lleven a cabo, sean positivos, productivos y que se respete el tiempo establecido (time-box) para cada uno de ellos.

Eventos de SCRUM

El Sprint es un contenedor para todos los eventos

Sprint

Son eventos de longitud fija de un mes o menos para crear consistencia. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior.

Todo el trabajo necesario para alcanzar el objetivo del producto.

Durante el Sprint:

- No se hacen cambios que pongan en peligro el Objetivo Sprint;
- La calidad no disminuye;
- El trabajo pendiente del producto se refina según sea necesario;
- El alcance se puede clarificar y renegociar con el Propietario del Producto a medida que se aprende más.

1. Sprint Planning (Planificación del Sprint)

Inicia el Sprint y establece el trabajo que se realizará. Participa todo el Scrum Team, que define colaborativamente:

- ¿Por qué? Se establece el Sprint Goal (Objetivo del Sprint), que expresa el valor que se busca entregar.
- ¿Qué? Los Developers seleccionan los elementos del Product Backlog que consideran que pueden completar durante el Sprint.
- ¿Cómo? Los Developers planifican cómo realizar el trabajo seleccionado, generalmente descomponiéndolo en tareas más pequeñas.

Resultado: Sprint Backlog, compuesto por el Sprint Goal, los elementos seleccionados y el plan para desarrollarlos.

2. Daily Scrum (Scrum Diario)

Reunión diaria de los Developers para inspeccionar el progreso hacia el Sprint Goal y adaptar el Sprint Backlog según sea necesario.

- Se realiza todos los días laborables, en el mismo horario y lugar.
- Permite identificar impedimentos, mejorar la comunicación y ajustar el trabajo del día siguiente.
- Los Developers eligen libremente cómo organizar la reunión, siempre que esté enfocada en el objetivo del Sprint.
- No es una reunión para informar el estado del trabajo al Scrum Master.

3. Sprint Review (Revisión del Sprint)

Su objetivo es inspeccionar el resultado del Sprint y determinar futuras adaptaciones del producto.

- Participan el Scrum Team y los stakeholders (interesados).
- Se presentan y revisan los resultados del trabajo realizado.
- Se obtiene feedback de los interesados.
- Se analiza el progreso hacia el Product Goal.
- Se puede adaptar el Product Backlog según lo aprendido y las nuevas necesidades.

No debe limitarse a una demostración del producto, sino que es una sesión de trabajo colaborativa.

4. Sprint Retrospective (Retrospectiva del Sprint)

Su objetivo es mejorar la calidad y eficacia del Scrum Team, inspeccionando cómo se trabajó durante el Sprint.

- Participa todo el Scrum Team.
- Se analizan las personas, interacciones, procesos, herramientas y Definition of Done.
- Se identifica qué funcionó bien, qué problemas surgieron y cómo se resolvieron.
- Se proponen mejoras para los próximos Sprints.
- Las mejoras más importantes pueden incorporarse al siguiente Sprint Backlog.

Es el último evento del Sprint.

5. Product Backlog Refinement (Refinamiento)

Es una actividad continua que busca preparar los elementos del Product Backlog para que puedan seleccionarse en futuros Sprints.

Evento	Pregunta clave	Enfoque principal
Sprint Planning	¿Qué vamos a lograr y cómo?	Planificar el Sprint.
Daily Scrum	¿Cómo estamos avanzando?	Inspeccionar y adaptar el trabajo diario.
Sprint Review	¿Qué producto construimos?	Evaluar el resultado y obtener feedback.
Sprint Retrospective	¿Cómo trabajamos?	Mejorar la forma de trabajar del equipo.
Refinamiento	¿Qué debemos preparar para después?	Aclarar y preparar futuras historias.

Timebox en SCRUM



Artefactos de SCRUM

Los artefactos de Scrum representan trabajo o valor y buscan maximizar la transparencia, permitiendo que todos tengan la misma información para inspeccionar y adaptar el trabajo.

1. Product Backlog (Pila del Producto)

Es una lista emergente y ordenada de todo lo necesario para mejorar el producto. Constituye la única fuente de trabajo del Scrum Team.

- Contiene los elementos o funcionalidades que se necesitan desarrollar.
- Evolucionan constantemente a medida que aparecen nuevas necesidades.
- El Product Owner es responsable de su gestión y ordenamiento.
- Mediante el refinamiento, los elementos se dividen y detallan para que sean más pequeños, claros y precisos.
- Los Developers son responsables de estimar el tamaño de los elementos.

Compromiso: Product Goal (Objetivo del Producto)

Describe el estado futuro que se desea alcanzar con el producto. Es el objetivo a largo plazo que orienta el trabajo del Scrum Team.

Se debe cumplir o abandonar un Product Goal antes de asumir otro.

2. Sprint Backlog (Pila del Sprint)

Es el plan de trabajo creado por y para los Developers, que contiene lo necesario para alcanzar el objetivo del Sprint.

Está compuesto por tres elementos:

- Por qué: Sprint Goal (objetivo del Sprint).
- Qué: elementos del Product Backlog seleccionados para el Sprint.
- Cómo: plan para desarrollar esos elementos y entregar un incremento.

Se crea durante el Sprint Planning y se actualiza a lo largo del Sprint según lo aprendido. Debe tener suficiente detalle para permitir inspeccionar el progreso durante el Daily Scrum.

Compromiso: Sprint Goal (Objetivo del Sprint)

Es el único objetivo que se busca alcanzar durante el Sprint. Proporciona enfoque y coherencia al trabajo del equipo.

El alcance del Sprint Backlog puede renegociarse con el PO a medida que se aprende más, siempre que no se ponga en peligro el Sprint Goal.

3. Increment (Incremento)

Es una versión funcional y utilizable del producto que representa un avance concreto hacia el Product Goal.

- Cada incremento se suma a los anteriores y debe funcionar correctamente con ellos.
- Debe estar completamente verificado y cumplir con la Definition of Done (DoD).
- Pueden generarse varios incrementos dentro de un mismo Sprint.
- Puede entregarse a los interesados antes de finalizar el Sprint; no es necesario esperar a la Sprint Review.

Compromiso: Definition of Done (DoD) – Definición de Hecho

Es la descripción formal de las condiciones de calidad que debe cumplir un incremento para considerarse terminado.

Artefacto	¿Qué representa?	Compromiso asociado
Product Backlog	Todo el trabajo necesario para mejorar el producto.	Product Goal
Sprint Backlog	Trabajo seleccionado y planificado para el Sprint actual.	Sprint Goal
Increment	Producto funcional y utilizable generado durante el Sprint.	Definition of Done

El triple filtro de SCRUM: Output, Outcome y Ready

Son definiciones que permiten establecer cuándo una User Story está lista para desarrollarse, cuándo está terminada y cuándo genera el valor esperado.

Concepto	Definición
Definition of Ready (DoR)	Establece las condiciones que debe cumplir una US para estar suficientemente clara, comprendida y estimada, de modo que pueda incorporarse a un Sprint.
Output Done	Establece las condiciones que debe cumplir una funcionalidad para considerarse terminada desde el punto de vista de la construcción y calidad del producto (desarrollada, probada, integrada, etc.).
Outcome Done	Establece las condiciones para comprobar que la funcionalidad entregada produjo el resultado o valor esperado para los usuarios o el negocio.

Ejemplo: aplicación para solicitar taxis

- Ready: la US para visualizar la ubicación del taxi está clara, tiene criterios de aceptación y puede estimarse.
- Output Done: la funcionalidad está desarrollada, integrada, probada y funciona correctamente.
- Outcome Done: se comprueba que los pasajeros utilizan la funcionalidad y que esta reduce la incertidumbre sobre la llegada del taxi.

- Ready: ¿Está lista para desarrollarse?
- Output done: ¿contruimos correctamente la funcionalidad?
- Outcome done: ¿Logramos el resultado esperado?

Importante: que una funcionalidad esté terminada técnicamente (Output Done) no garantiza que genere el valor esperado (Outcome Done). Para comprobarlo, necesitamos observar su utilización y medir los resultados.

Unidad 3 - Software Configuration Management (SCM)

- Gestión de Configuración del Software
- Es una disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, relevante para el producto a lo largo de su ciclo de vida. Surge para enfrentar la crisis de problemas de software, aplicando dirección y monitoreo (administrativo y técnico) a:
 - o Identificar características técnicas y funcionales de ítems de configuración;
 - o Documentar características técnicas y funcionales de ítems de configuración;
 - o Controlar los cambios de esas características identificadas y documentadas;
 - o Registrar y reportar estos cambios;
 - o Verificar correspondencia con los requerimientos (trazabilidad).
- Su propósito es resolver problemas de diferente índole, a través del establecimiento y el mantenimiento de la integridad del producto de software a lo largo de todo su ciclo de vida.
- Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.
- La integridad es el medio por el cual podemos garantizar que el producto a entregar tiene la calidad correspondiente. Y nos garantiza un nivel mínimo de confiabilidad.
- El problema de la calidad es que es subjetiva y se suma a que el software es intangible, por lo tanto, es muy difícil medir si la calidad que responde al cumplimiento de las expectativas del cliente realmente se verifica.
- La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.
- Decimos que se mantiene la integridad de un producto de software cuando:
 - o Satisface las necesidades de usuario: hace lo que el usuario espera que haga;
 - o Permite la rastreabilidad durante todo su ciclo de vida: existen vínculos o conexiones entre los ítems de configuración (deben ser definidos desde un primer momento del ciclo de vida del producto), que permiten analizar en donde impactará un cambio en un ítem. De esta forma, se puede determinar cómo impactará cada cambio de requerimientos a través de la trazabilidad. A mayor cantidad de vínculos → mayor información → mayor trazabilidad → mayor costo (analizar la relación costo-beneficio);
 - o Satisface criterios de performance y RNF;
 - o Cumple con expectativas de costo: este apartado incluye la satisfacción del equipo de proyecto, todos los miembros deben estar conformes

Algunos de sus problemas son:

- Pérdida de componentes;
- Pérdida de cambios (la versión del componente no es la última);

- Regresión de fallas;
- Doble mantenimiento;
- Superposición de cambios;
- Cambios no validados.

¿Cuál es el problema que se está tratando de solucionar?

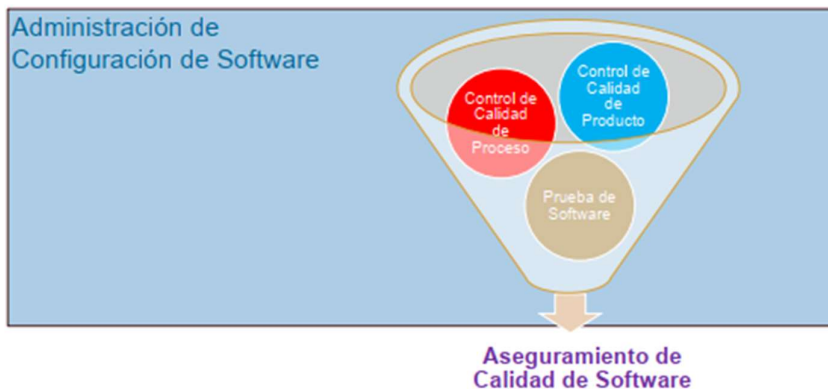
El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo. Entonces lo que buscamos con la gestión de configuración del software es tratar de que estas cosas no pasen, buscando que el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento. Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión. Y todas las personas se enteren de los cambios.

Cambios en el software

Tienen su origen en:

- Cambios del negocio y nuevos requerimientos → el software genera constantemente cambios, a medida que se usa
- Soporte de cambios de productos asociados
- Reorganización de las prioridades de la empresa por crecimiento
- Cambios en el presupuesto
- Defectos encontrados a corregir
- Oportunidades de mejora

Disciplinas de soporte del software



Conceptos generales

Ítem de configuración

Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida (ya sea del producto o proyecto, dependiendo del artefacto). Es decir, es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener. Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos).

Los ítems, dependiendo de su naturaleza, pueden durar lo que dure el ciclo de vida del proyecto, producto o sprint.

Algunos ejemplos: documentos de diseño, código fuente, código ejecutable, prototipo de interfaz, manual de usuario, ERS, arquitectura del software, casos de prueba, íconos

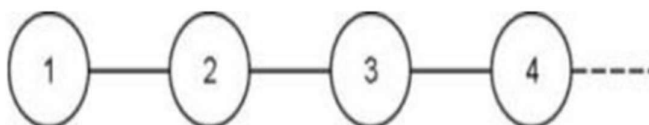
- Todo elemento de software que puedan ser guardados en un disco duro de la computadora
- Las 3 disciplinas generan ítems de configuración
- Pieza de software que vos necesitas guardar en un repositorio. Posee nombre único y versión asociada

Más ejemplos

- | | |
|-----------------------------|-------------------------------|
| ❖ Plan de CM | ❖ Planes de Iteración |
| ❖ Propuestas de Cambio | ❖ Estándares de codificación |
| ❖ Visión | ❖ Casos de prueba |
| ❖ Riesgos | ❖ Código fuente |
| ❖ Plan de desarrollo | ❖ Gráficos, iconos, ... |
| ❖ Prototipo de Interfaz | ❖ Instructivo de ensamble |
| ❖ Guía de Estilo de IHM | ❖ Programa de instalación |
| ❖ Manual de Usuario | ❖ Documento de despliegue |
| ❖ Requerimientos | ❖ Lista de Control de entrega |
| ❖ Plan de Calidad | ❖ Formulario de aceptación |
| ❖ Arquitectura del Software | ❖ Registro del proyecto |
| ❖ Plan de Integración | |

Versión

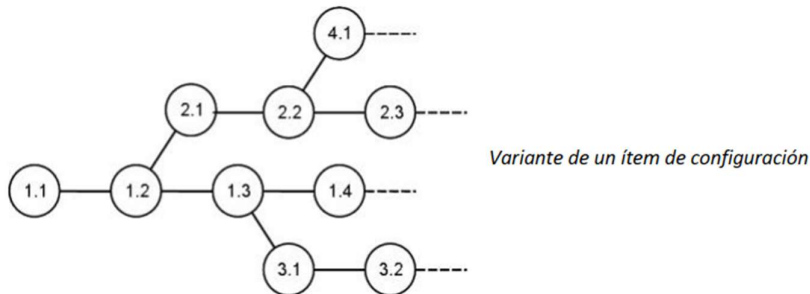
- Instancia de un ítem de configuración que difiere de otras instancias de este ítem. Las versiones se identifican unívocamente. Controlar una versión refiere a la evolución de un único ítem de configuración.
- Se pueden asociar atributos a cada versión (que pueden ser datos sencillos como un nro. de versión asociado a cada objeto. Cada versión de software es una colección de elementos de configuración (ECS), como ser código fuente, documentos, datos).
- La versión es un punto particular en el tiempo de ese ítem de configuración (es un estado).
- Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.
- Control de versiones → se refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado.
- La evolución puede representarse gráficamente en forma de grafo.



Evolución lineal de un ítem de configuración

Variante

- Versión de un ítem de configuración (o de la configuración) que evoluciona por separado.
- Representan configuraciones alternativas.
- Permiten satisfacer requerimientos en conflicto, como por ejemplo la compatibilidad del producto con diferente hardware o sistemas operativos.
- Si la diferencia que existe entre las instancias de un mismo ítem de configuración es muy pequeña, no se denomina versión, sino que se la conoce como variante.
- Está ligado a trabajar con Branches, donde el producto adopta distintas configuraciones, dependiendo de múltiples factores (problemas, SO donde se ejecutará, etc.).



Configuración de software

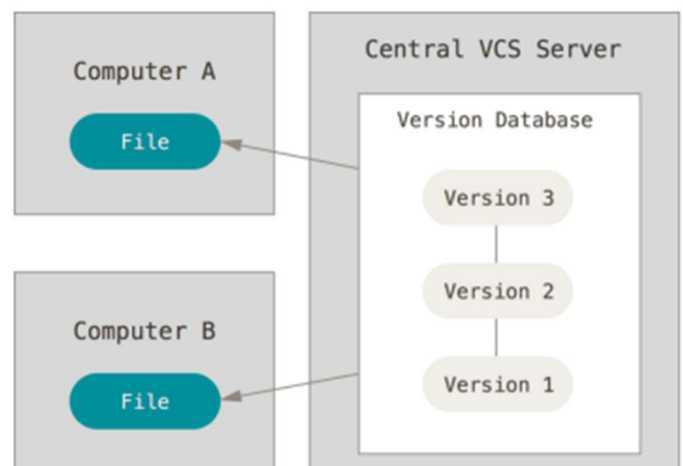
- Conjunto de todos los ítems de configuración con su versión específica. Define una foto de los ítems de configuración en un momento de tiempo determinado.
- Es el conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar.

Repositorio

- Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos.
- Puede ser una o varias bases de datos.
- Tiene una estructura para mantener el orden y la integridad. El que tenga una estructura ayuda a la seguridad, los controles de acceso, las políticas de Backup y todo aquello que se aplica sobre un repositorio.

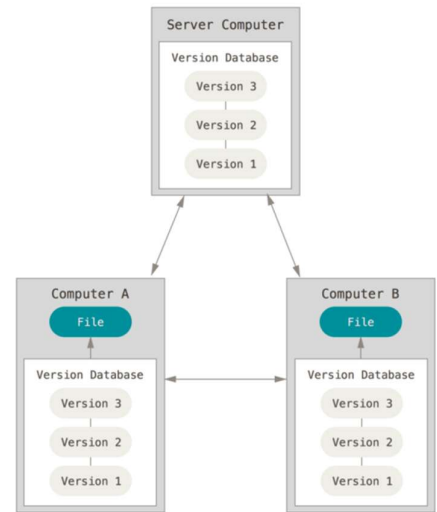
1) Repositorios centralizados:

- o Está caracterizado porque un servidor contiene todos los archivos con sus versiones.
- o Los administradores tienen un mayor control sobre el repositorio.
- o Tiene como desventaja que si falla el servidor, todo lo positivo se cae.



2) Repositorios descentralizados:

- Está caracterizado porque cada cliente tiene una copia exactamente igual del repositorio completo.
- Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”
- Posibilita otros workflows no disponibles en el modelo centralizado.
- La desventaja que podemos mencionar es que es más complicado llevar un control sobre el mismo.



Release

Entrega de un sistema que se libera para su uso a los clientes u otros usuarios de la organización.

Rama (Branch)

Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos:

- Experimentación
- Resolución de errores en el desarrollo
- Desarrollar un mismo software para diferentes plataformas

Existe una rama principal (trunk, master)

Integración de ramas (Merge)

Se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones, en cada una de ellas. Una buena práctica es mantener la rama principal como la versión estable, y fusionar los cambios hacia ella. En caso de no integrarse a la Main, las ramas deberían descartarse.

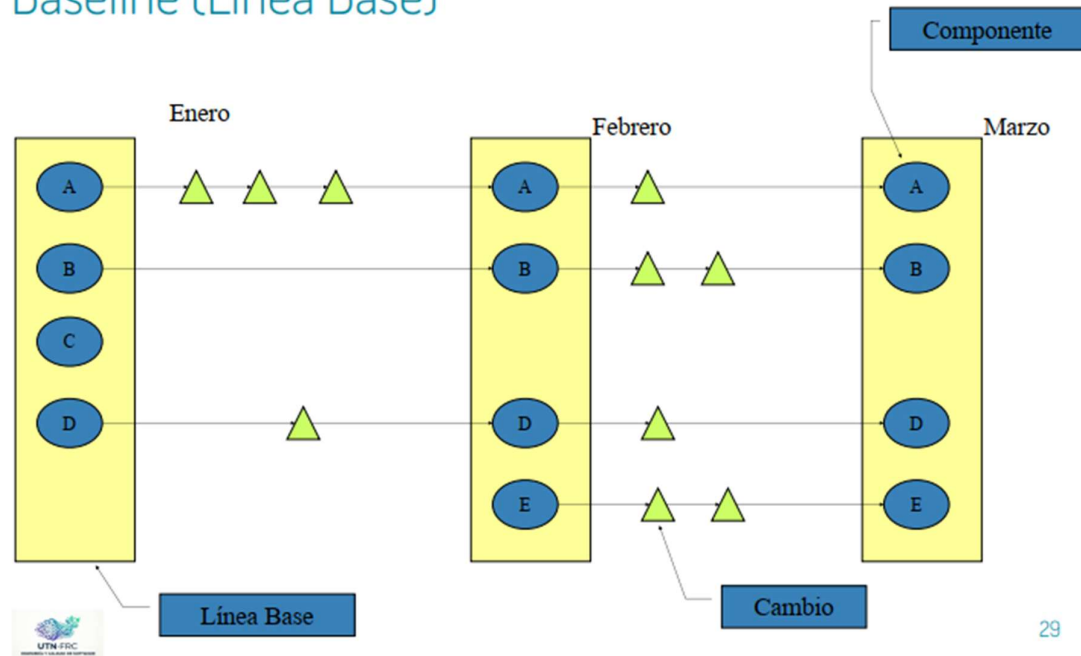
El rol de las líneas base y su administración:

Es un conjunto de ítems de configuración que han sido construidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores. Esto es lo mismo que decir que es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros.

Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags” (etiquetas).

Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el Comité de Control de Cambios, que permite definir el procedimiento a seguir para manejar peticiones de cambios, y en caso de aceptarse, acordar nuevamente los parámetros y comunicar los cambios a todo el equipo, para que tomen la nueva línea base como modelo a seguir.

Baseline (Línea Base)



29

En enero se establece una línea base compuesta por los componentes A, B, C y D. Durante el desarrollo se realizan modificaciones, se elimina C y se incorpora E. En febrero se aprueba una nueva línea base con A, B, D y E. Posteriormente, estos componentes continúan modificándose y en marzo se establece otra línea base.

Las líneas base permiten mantener un registro de las configuraciones aprobadas del software, identificar los cambios realizados y recuperar configuraciones anteriores.

¿Para qué sirve una línea base?

Es para tener un punto de referencia, pero también sirve para hacer Rollback (volver a una versión o configuración anterior del software o revertir un cambio) o saber qué se pone en producción. Nos permite saber cuál era la última situación estable en un momento de tiempo y cómo se fue evolucionando. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.

2 tipos:

- 1) Operacionales → contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release. Es la línea base de productos que han pasado por un control de calidad definido previamente.
- 2) De especificación o documentación → son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento. Son de requerimientos, diseño.
 - o Definiciones y decisiones que se van tomando a lo largo del proyecto (documentación sobre producto/proyecto)

Líneas base durante el ciclo de vida



A medida que avanza el desarrollo del software, se establecen líneas base sobre los entregables aprobados de cada etapa (planificación, requisitos, diseño, codificación y pruebas), permitiendo tener puntos referencia, controlar los cambios y conservar trazabilidad de los elementos de configuración

Planificación de la gestión de configuración del software

Incluye:

- Reglas de nombrado de los ítems de configuración;
- Herramientas para utilizar en SCM;
- Roles e integrantes del Comité de Control de Cambios;
- Procedimientos formales de cambios;
- Procesos de auditoría;
- Estructura del repositorio;
- SCM para software externo (opcional);
- Cómo se hará el control de cambios;
- Registros que deben mantenerse;
- Tipos de documentos.

Consideraciones:

- Debe hacerse tempranamente;
- Se deben definir los documentos que se administrarán;
- Todos los productos del proceso deben administrarse.

Actividades Fundamentales de la SCM

Identificación de Ítems de Configuración

- Identificación unívoca de cada ítem de configuración
- Convenciones y reglas de nombrado
- Definición de la Estructura del Repositorio
- Ubicación dentro de la estructura del repositorio

Clasificación de los Ítems:



- 1) **Ítems de producto** → Elementos que forman parte del producto de software y describen sus características, estructura y funcionamiento. Tienen el ciclo de vida más largo, y se mantienen mientras el producto exista
- 2) **Ítems de proyecto** → Elementos utilizados para planificar, organizar, gestionar y controlar el desarrollo del proyecto.
- 3) **Ítems de iteración** → Elementos generados o utilizados durante una iteración específica del desarrollo, que permiten planificar y realizar el seguimiento del trabajo de ese período.

Control de cambios

Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios. Es decir que el control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia.

Los ítems de configuración que no forman parte de la línea base uno los trabaja libremente, no hay que pedirle al comité de cambios

Comité de control de cambios:

Este proceso es llevado a cabo por el comité de control de cambios, el cual se reúne para autorizar la creación y cambios sobre la línea base. Forman parte de dicho comité los interesados en evaluar y en enterarse del cambio, decidiendo si lo aceptan o no. El líder del proyecto, el analista, el arquitecto e incluso el cliente pueden constituirlo.

Auditorías de configuración

Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten “congelar” y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM.

La auditoría de configuración se hace mientras el producto se está construyendo y su objetivo es que el producto tenga calidad e integridad.

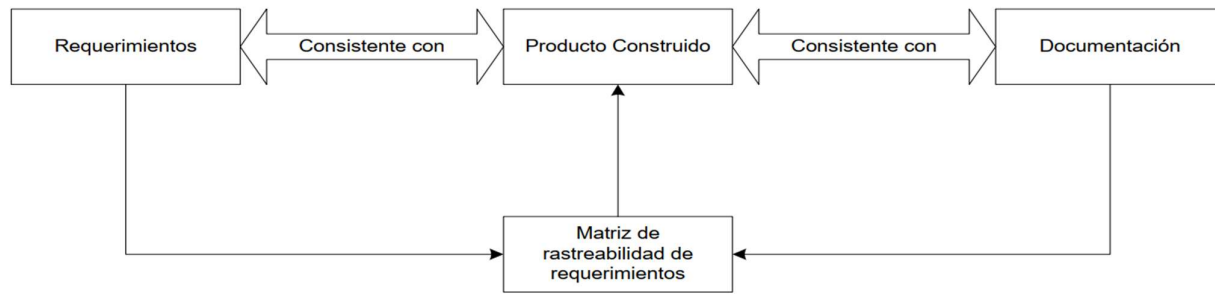
- Es objetiva cuando hay independencia de criterio, por lo que el auditor no tiene ningún condicionamiento respecto de lo auditado.
- Es independiente cuando no depende jerárquica, funcional ni salarialmente del equipo o elemento a auditar.

Se planifican en el Plan de Gestión de Configuración → sé en que momento tengo que hacer una auditoría (Ejemplo → si estoy trabajando con el proceso de desarrollo, después de cada workflow)

Tipos de auditorías

Auditoría Funcional de Configuración

Auditoría Física de Configuración



- 1) Auditoría física o de contingencia → se realiza primero. Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración, asegurando que la documentación que se entregará es consistente con el código desarrollado. Si esto no pasa, la auditoría funcional no se realiza.
 - Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo que iban a estar guardados.
 - Lo que se audita es una línea base.
 - Hace verificación.
- 2) Auditoría funcional de configuración → Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos. Antes de comenzar, pregunta si está disponible el reporte de la auditoría de configuración física. Ya que, si la auditoría física no sale bien, la auditoría funcional no se hace.
 - Hace validación (que el código implementa únicamente y todos los requerimientos de la ERS)

Procesos a los que sirve:

- 1) Validación → se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, es decir, lo que el cliente necesita.
- 2) Verificación → implica asegurar que un producto cumple con los objetivos definidos en la documentación de líneas base. Todas las funciones son llevadas a cabo con éxito y los test cases tengan status "ok" o bien consten como "problemas reportados" en la nota de reléase. Es decir, que se cumpla lo definido para cada ítem de configuración en la documentación de una línea base.

Informes de Estado

- Se ocupa de mantener los registros de la evolución del sistema.
- Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos.
- Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.

Permite responde a preguntas como:

- a. ¿Cuál es el estado del ítem?
- b. ¿La propuesta de cambio fue aceptada o rechazada por el comité?
- c. ¿Qué versión de ítem implementa un cambio de una propuesta de cambio aceptada?
- d. ¿Cuál es la diferencia entre dos versiones de un mismo ítem?

Gestión de configuración en ambientes ágiles

En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo.

En orden con la caracterización de los equipos ágiles, decimos que la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.

Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles. Todos los procesos definidos establecidos en la gestión de configuración del enfoque tradicional son relativizados en agile. Por ejemplo, no existe un comité para el proceso forma de control de cambios.

Relación con Manifiesto Ágil

Podemos decir que va de la mano con el manifiesto ágil porque recordemos que se buscaba estar siempre preparados para el cambio y justamente, la gestión de configuración busca responder a los cambios y mantener la integridad del producto.

La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto.

Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto.

Características

- 1) Dada la naturaleza de los requerimientos ágiles, la SCM responde a los cambios en lugar de evitarlos.
- 2) La automatización debe ser aplicada donde sea posible. Esto colabora con la entrega frecuente y rápida de software (integración continua).
- 3) Provee retroalimentación continua sobre calidad, estabilidad e integridad.
- 4) Las actividades de SCM se encuentran embebidas en las demás tareas para alcanzar el objetivo del sprint.
- 5) Es responsabilidad de todo el equipo, por lo tanto, requiere capacitación.
- 6) 6Adopción de las prácticas continuas.